EXPERIMENT-3

NAME:NIMALAN M ROLL NO:240701362

1. COMMAND LINE INTERFACE:

PYTHON CODE:

```
import requests


print("=== CLI WEATHER CHECKER ===")


city = input("Enter city name: ")


# Fixed coordinates (Chennai weather demo)

lat, lon = 13.08, 80.27


url = f"https://api.open-
meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true"


data = requests.get(url).json()


weather = data['current_weather']


print("\nWeather for:", city)

print("Temperature:", weather['temperature'], "°C")

print("Wind Speed:", weather['windspeed'], "km/h")
```

OUTPUT:

```
PS D:\ent\exp 3 uid> python -u "d:\ent\exp 3 uid\cli_weather.py"
=== CLI WEATHER CHECKER ===
Enter city name: Chennai

Weather for: Chennai
Temperature: 23.8 °C
Wind Speed: 3.9 km/h
PS D:\ent\exp 3 uid>
```

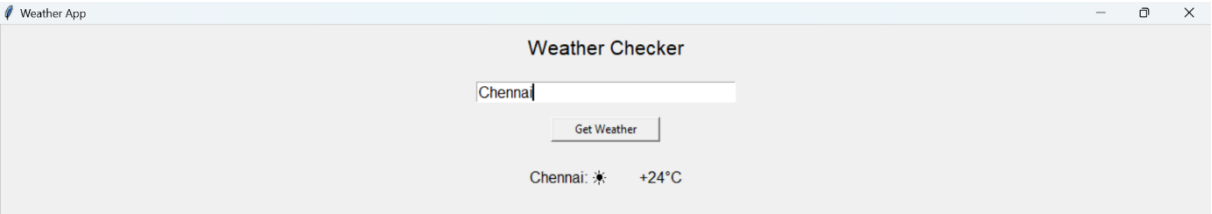
2. GRAPHICAL USER INTERFACE:

PYTHON CODE:

```
import tkinter as tk

import requests

import threading


# --- Functions ---


def get_weather():

    """Fetch weather for the city entered in the textbox."""

    def fetch():

        city = city_entry.get().strip()

        if not city:

            result_label.config(text="Enter a city name")

            return

        try:

            url = f"https://wttr.in/{city}?format=3"

            response = requests.get(url, timeout=5)

            result_label.config(text=response.text)

        except Exception as e:

            result_label.config(text=f"Error fetching weather: {e}")


    # Run in separate thread to avoid freezing the GUI

    threading.Thread(target=fetch).start()


# --- GUI Setup ---


root = tk.Tk()

root.title("Weather App")

root.geometry("450x250")

root.lift()

root.attributes('-topmost', True)
```

```
root.after(1000, lambda: root.attributes('-topmost', False))

# Heading
tk.Label(root, text="Weather Checker", font=("Arial", 16)).pack(pady=10)

# City textbox
city_entry = tk.Entry(root, width=30, font=("Arial", 12))
city_entry.pack(pady=10)

# Button
tk.Button(root, text="Get Weather", command=get_weather, width=15).pack(pady=5)

# Result label
result_label = tk.Label(root, text="", wraplength=400, font=("Arial", 12))
result_label.pack(pady=20)

# Start GUI
root.mainloop()
```

OUTPUT:



3. VOICE USER INTERFACE:

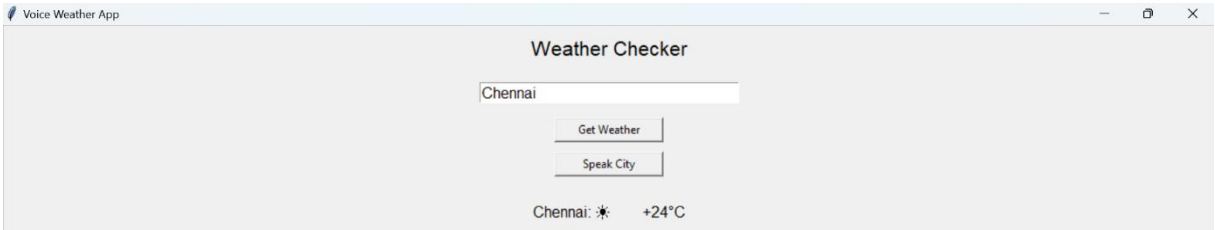
PYTHON CODE:

```
import tkinter as tk
import requests
import speech_recognition as sr
import threading

# --- Functions ---

def get_weather(city=None):
    """Fetch weather for a city."""
    def fetch():
        nonlocal city
        if city is None:  # If not passed, get from entry
            city = city_entry.get()
        if not city:
            result_label.config(text="Enter a city name")
            return
        try:
            url = f"https://wttr.in/{city}?format=3"
            response = requests.get(url)
            result_label.config(text=response.text)
        except Exception as e:
            result_label.config(text=f"Error: {e}")

    threading.Thread(target=fetch).start()  # Run in separate thread to avoid freezing GUI

def get_city_voice():
    """Use microphone to get city name."""
    recognizer = sr.Recognizer()
    mic = sr.Microphone()
```

```
    result_label.config(text="Listening...")

    def listen():
        try:
            with mic as source:
                recognizer.adjust_for_ambient_noise(source)
                audio = recognizer.listen(source, timeout=5)
            city_name = recognizer.recognize_google(audio)
            city_entry.delete(0, tk.END)
            city_entry.insert(0, city_name)
            get_weather(city_name)
        except sr.WaitTimeoutError:
            result_label.config(text="No speech detected, try again.")
        except sr.UnknownValueError:
            result_label.config(text="Could not understand audio.")
        except sr.RequestError:
            result_label.config(text="Could not request results from Google API.")

    threading.Thread(target=listen).start()  # Run in thread to avoid freezing GUI

# --- GUI ---

root = tk.Tk()
root.title("Voice Weather App")
root.geometry("450x300")
root.lift()
root.attributes('-topmost', True)
root.after(1000, lambda: root.attributes('-topmost', False))

# Heading
tk.Label(root, text="Weather Checker", font=("Arial", 16)).pack(pady=10)

# City textbox
```

```
city_entry = tk.Entry(root, width=30, font=("Arial", 12))
city_entry.pack(pady=10)

# Buttons
tk.Button(root, text="Get Weather", command=get_weather, width=15).pack(pady=5)
tk.Button(root, text="Speak City", command=get_city_voice, width=15).pack(pady=5)

# Result label
result_label = tk.Label(root, text="", wraplength=400, font=("Arial", 12))
result_label.pack(pady=20)

root.mainloop()
```

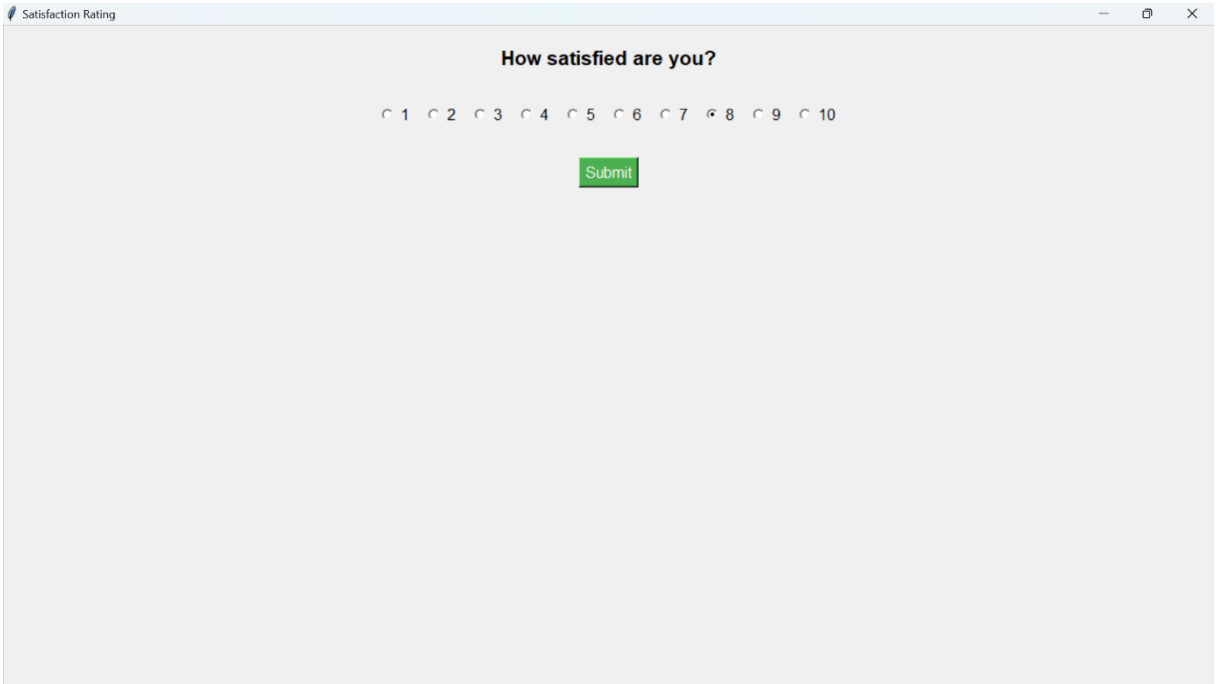
Output:



4. USER SATISFACTION COMPARISON:

PYTHON CODE:

```
def survey():
    print("Rate your satisfaction with the following interfaces (1-5):")

    # Get user input for each interface
    try:
        cli_satisfaction = int(input("CLI (1-5): "))
        gui_satisfaction = int(input("GUI (1-5): "))
        vui_satisfaction = int(input("VUI (1-5): "))

        # Ensure valid ratings
        if not (1 <= cli_satisfaction <= 5 and 1 <= gui_satisfaction <= 5 and 1 <= vui_satisfaction <= 5):
            print("Please enter ratings between 1 and 5 only.")
            return

        # Display the ratings
        print("\nYour satisfaction ratings:")
        print(f"CLI: {cli_satisfaction}")
        print(f"GUI: {gui_satisfaction}")
```

```
    print(f"VUI: {vui_satisfaction}")

        # Calculate the average satisfaction
        avg_satisfaction = (cli_satisfaction + gui_satisfaction + vui_satisfaction) / 3
        print(f"\nAverage Satisfaction Score: {avg_satisfaction:.2f}")

    except ValueError:
        print("Invalid input! Please enter numbers between 1 and 5.")

# Run the survey function
if __name___== "__main__":
    survey()
```

Output: